

C++ Binary Fixed-Point Arithmetic

ISO/IEC JTC1 SC22 WG21 P0106R0 - 2015-09-27

Lawrence Crowl, Lawrence@Crowl.org

[Introduction](#)

[Fixed-Point versus Integer](#)

[Fixed-Point versus Floating-Point](#)

[Prior Art](#)

[Proposal Outline](#)

[Basic Types](#)

[Basic Operations](#)

[Construction and Assignment](#)

[Literals](#)

[Examples](#)

[Alpha Blending](#)

[Performance Discussion](#)

[Proposal Details](#)

[Type Signatures](#)

[Operations](#)

[Revisions](#)

[References](#)

Introduction

C++ supports integer arithmetic and floating-point arithmetic, but it does not support fixed-point arithmetic. We propose support for fixed-point arithmetic via standard library facilities.

Fixed-Point versus Integer

In C and C++, the basic integer types have several problematic behaviors.

- Signed integer arithmetic overflow results in undefined behavior. Pre-emptively checking for overflow is challenging and tedious, so programmers very rarely do so. As a result, most programmers simply assume it will not happen.
- Unsigned integer arithmetic overflow results in well-defined behavior, but that behavior is not always desirable. Again, pre-emptively checking for overflow is challenging and tedious.
- C/C++ signed integers promote to unsigned integers, which is the exact opposite of the relationship between their mathematical analogs; (unsigned) cardinal numbers are a strict subset of (signed) integral numbers. Because of this promotion, it is difficult to prevent signed numbers from being used in places where they are not intended.
- C/C++ integer ranges are platform-specific, which generally either binds programs to a platform or requires considerable care in production of portable code.

Fixed-Point versus Floating-Point

Fixed-point arithmetic is better than floating-point arithmetic in several domains.

- Problems with a constrained range can use bits for resolution that would otherwise have gone to an exponent. Examples include Mandelbrot set computation and angular position.
- Problems with a constrained resolution can use smaller representation than a single-precision floating-point. An example is a pixel color value.
- Low-cost or low-power systems may not provide floating-point hardware. For these systems, fixed-point arithmetic provides a much higher performance alternative to software-implemented floating-point arithmetic for many problems.

Prior Art

The popular computing literature abounds with articles on how to use integers to implement fixed-point arithmetic. However, manually writing fixed-point arithmetic with integer arithmetic is tedious and prone to error. Direct support is desirable.

ISO/IEC TR 18037 [E] provides fixed-point support in the C programming language. However, this support is not general; only a few possible radix positions are supported. The feature is essentially limited to the digital signal processing domain.

Likewise, software implementations of fixed-point arithmetic in C and C++, e.g. libfixmath [F], are also not general as the support only a limited number of radix positions.

Since the first version of this paper, John McFarlane has written a fixed-point proposal [M]. It is based on an integer type with a constant scaling factor. It has the virtues of an unlimited number of radix positions and the capacity to mix different radix positions in operations. Unfortunately, it does not track range and so provides limited assistance in avoiding overflow. It also provides no mechanism to control rounding.

The programming languages Ada [A], COBOL [B], CORAL 66 [C1] [C2], JOVIAL [J], and PL/I [P] provide direct support for fixed-point arithmetic.

Proposal Outline

We propose extending the standard library to provide general purpose binary fixed-point arithmetic. We anticipate that these library components will be used for the manipulation of program data variables, and not program control variables. That is, we do not view the proposal as a replacement for index or size variables.

The design relies on generally 'safe' defaults, but with additional explicit controls to match particular application domains or to enable more efficient execution.

Our proposal requires no new hardware, and is implementable as a pure library. Indeed, much of that implementation already exists. However, some operations could be substantially faster with direct hardware support. A rounding right shift is at the top of the list.

Basic Types

The fixed-point library contains four class templates generally intended as variable types. They are `cardinal` and `integral` for integer arithmetic, and `nonnegative` and `negatable` for fractional arithmetic. In addition, there are four class templates generally intended as types for values or const parameters. They are `card_val` and `intg_val` for integer arithmetic, and `nonn_val` and `negt_val` for fractional arithmetic.

These types have a range specified by an integer. The range of an unsigned number n is $0 \leq n < 2^g$ where g is the range parameter. The range of a signed number n is $2^g < n < 2^g$. Note that the range interval is half-open for unsigned numbers and open for signed numbers. That is *signed values are symmetric around zero*. For example, `cardinal<8>` has values n such that $0 \leq n < 256$ and `integral<8>` has values n such that $-256 < n < 256$.

The fractional types have a resolution specified by an integer. The resolution of a fractional number n is 2^s , where s is the resolution parameter. For example, `negatable<8, -4>` has values n such that $-256 < n < 256$ in increments of $2^{-4} = 1/16$.

Both range and resolution parameters may be either positive or negative. The number of significant bits is $g-s$. This specification enables representing both very small and very large values with few bits. In any event, the range must be greater than the resolution, that is $g > s$.

Basic Operations

The basic arithmetic operations are addition, subtraction, multiplication and division. When mixing operands of different template class types, `cardinal` types will promote to the other types, and the other types will promote to `negatable` type. The effect is that unsigned fixed-point types promote to signed fixed-point types. There are notable exceptions. Negation and subtraction on unsigned types yields a signed type. Comparison across types is direct; there is no conversion beforehand.

In general, the range and resolution of the result of basic operations are large enough to hold the mathematical results. The following table shows the range and resolution for the results of basic operations on fractional types. The `$` operator returns the range parameter. The `@` operator returns the resolution parameter.

operation	result range	result resolution
<code>a+b</code>	<code>max(\$a,\$b)+1</code>	<code>min(@a,@b)</code>
<code>a-b</code>	<code>max(\$a,\$b)+1</code>	<code>min(@a,@b)</code>
<code>a*b</code>	<code>\$a+\$b</code>	<code>@a+@b</code>
<code>a/b</code>	<code>\$a-@b</code>	<code>@a+\$b</code>

Overflow in template argument computation is undefined behavior. In practice, overflow is unlikely to be a significant problem

because even small machines can represent numbers with thousands of bits and because compiler can diagnose overflow in template arguments.

The special case in the operations above is division, where the mathematical result may require an infinite number of bits. The actual value must be rounded to a representable value. The above resolution is sufficient to ensure that if the mathematical result is not zero, the fixed-point result is not zero. Furthermore, assuming values have an error of one-half ULP, the defined resolution is close to the error bound in the computation.

When the computation is not exact, rounding will be to one of the two nearest representable values. The algorithm for choosing between these values is the rounding mode. Different applications desire different modes, so programmers may specify the rounding mode as in [P0105r0 Rounding and Overflow in C++](#).

Construction and Assignment

Since the range of intermediate values grow to hold all possible values, and variables have a static range and resolution, construction and assignment may need to reduce the range and resolution. Reducing the resolution is done with a rounding mode associated with the variable. When the dynamic value exceeds the range of variable, the assignment overflows.

When an overflow does occur, the desirable behavior depends on the application, so programmers may specify the overflow mode as in [P0105r0 Rounding and Overflow in C++](#).

Literals

User-defined literals that produce a parameter pack of characters may be able to provide a reasonable mechanism for writing literal values. That exploration has yet to be done.

In the meantime, we can get close with template functions that yield the appropriate fixed-point value based on an template `int` parameter. For example, the expression `to_cardinal<24>()` will produce a `cardinal` constant with a range just sufficient to hold the value 24. Likewise, the expression `to_nonnegative<2884, -4>()` will produce a `nonnegative` constant with a range and resolution just sufficient to hold the value $2884 \cdot 2^{-4}$.

Another approach is to rely on conversion from floating-point literals.

Examples

Alpha Blending

Consider the alpha blending of two RGBA pixels, `a` and `b`. To avoid redundancy, we will only show computation for the red color. The algorithm is somewhat complicated by the need to convert the `[0-255]` range of alpha representation to the `[0-1]` range needed for multiplication.

```
struct pixel {
    nonnegative<8,0> r, g, b, a; // Values 0 to 255 map to 0 to 1.
    // This mapping is not natural to binary, which prefers to map 256 to 1.
    // So, we scale alpha values to 0 to 1. We can leave rgb values alone.
    // Representing the type as a fixed-point with a resolution of one
    // gives us floating-point rounding rather than integer truncation.
};

pixel blend( pixel a, pixel b ) {
    pixel c; // output
    constexpr one = to_cardinal<1>();
    constexpr scale = to_nonnegative<255,0>();
    // compute output alpha
    auto a_a = a.a / scale;
    auto aia = (b.a / scale) * (one - a_a);
    auto c_a = a_a + aia;
    c.a = c_a * scale;
    // apply blending
    c.r = (a.r*a_a + b.r*aia) / c_a;
    // repeat with green and blue...
    return c;
};
```

Performance Discussion

There is substantial concern over the performance of the proposed library. The intent of the design is to provide well-behaved defaults, even at the cost of run-time performance. However, the efficiency of carefully crafted code using this library should

match or exceed performance of existing libraries. So, while recognizing 'default' cost, one must also understand the coding approaches that reduce that cost.

The library design is template heavy. It will have a higher compilation cost than other approaches. I consider this a price worth paying for much more robust code.

The library design minimizes the number of overflow detections. They occur only on assignment or conversion to a type with a smaller range.

Overflow detection can be completely eliminated with the appropriate overflow mode in a typedef.

The library design minimizes the number of scale_up operations. On addition and subtraction, they occur only where the two arguments do not have the same resolution. On assignment and conversion, they occur only where the destination has a finer resolution. Otherwise, they do not occur.

The library design minimizes the number of rounding scale_down operations. They occur only on assignment and conversion where the destination has a coarser resolution.

Division necessarily requires rounding.

While the default rounding mode is non-trivial, one can easily specify the cost-free truncation rounding mode by modifying the definition of a typedef.

Doubling representation on each multiplication or division could result in large representations. Normal fixed-point libraries scale down on each multiplication. Incurring a cost even when such scaling is not necessary for efficiency. By assigning a product to a variable of a smaller coarser resolution, that implicit scaling in other libraries becomes an explicit scaling in this proposed library. High-performance shops might even adopt a coding standard of having only one multiplication or division before assigning to a variable of known range and resolution.

Increasing the representation by one bit on each addition or subtraction can result in larger numbers. While this is true, most of that representation growth is free, because the representation is growing within a word. For example, increasing the representation from 18 bits to 19 bits has no practical effect. Indeed, on most modern systems, representations up to 32 bits have identical cost.

The specification of both range and resolution on the type encourages programmers to precisely bound the representation to the smallest number of bits. So, the growth discussed above starts from a smaller representation than would occur in other approaches.

The precise bounds have another benefit, which is that programmers can rely on the compiler to insert and track the 'slack' bits that prevent overflow. There will generally be fewer slack bits if the compiler does it than if the user does it.

Proposal Details

Type Signatures

The template class type signatures are as follows.

```
template<
    int Crng
>
class card_val;

template<
    int Crng,
    overflow Covf = overflow::exception
>
class cardinal;

template<
    int Crng
>
class intg_val;

template<
    int Crng,
    overflow Covf = overflow::exception
>
class integral;

template<
```

```
    int Crng,
    int Crsl
>
class nonn_val;

template<
    int Crng,
    int Crsl,
    round Crnd = rounding::tie_to_odd,
    overflow Covf = overflow::exception
>
class nonnegative;

template<
    int Crng,
    int Crsl
>
class negt_val;

template<
    int Crng,
    int Crsl,
    round Crnd = rounding::tie_to_odd,
    overflow Covf = overflow::exception
>
class negatable;
```

Operations

operations	types	notes
default construction	all	uninitialized
copy construction	all	identical value
value construction	from same or lower type	value subject to overflow and/or rounding
v.increment<overflow>();	cardinal and integral	value subject to overflow
v++; v--; ++v; --v		
-v	all	result is a signed type
!v	all	test for value != 0
~v	cardinal	invert bits, but still within range
v.scale_up<n>()	all	multiply by 2 ⁿ , where <i>n</i> >0
v.scale<n,r>()	all	multiply by 2 ⁿ , apply the rounding mode when <i>n</i> <0
v*u	all	multiplication
v.divide<round>(u)	cardinal and integral	integer division with the given roundng mode
v.divide<round>(u)	nonnegative and negatable	fractional division with the given roundng mode
v/u	cardinal and integral	v.divide<truncated>(u)
v/u	nonnegative and negatable	v.divide<class-specified rounding mode>(u)
v/u	nonnegative and negatable	v.divide<class-specified rounding mode>(u)
v%u	cardinal and integral	remainder
v+u	all	addition
v-u	all	subtraction, the result is always signed
comparisons	all	no value promotion/conversion, all comparisons are value-based
v&u; v^u; v u	cardinal	bitwise logical operations
v=u; a*=b; a/=b; a%=b; a+=b; a-=b;	all	(compound) assignment
v&=u; v^=u; v =u;	cardinal	bitwise logical compound assignment

Revisions

This paper revises [N3352 C++ Binary Fixed-Point Arithmetic](#)

- Add value types.

- The details of rounding and overflow have been moved to a separate paper.
- Add additional approaches to literals.
- Clean up alpha blending example.
- Add performance section.

References

- [A] *Ada Reference Manual*, <http://www.ada-auth.org/arm.html>
- [B] *ISO IEC JTC1/SC22/WG4 - COBOL*, <http://www.cobolstandard.info/wg4/wg4.html>
- [C1] *BS 5905:1980 Specification for computer programming language CORAL 66*, October 1980, <http://shop.bsigroup.com/ProductDetail/?pid=000000000000133933>
- [C2] *XGC CORAL 66 Language Reference Manual*, 2001, <http://www.xgc.com/manuals/xgc-c66-rm/book1.html>
- [E] *ISO/IEC TR 18037:2008: Programming languages -- C -- Extensions to support embedded processors*, http://www.iso.org/iso/iso_catalogue/catalogue_tc/catalogue_detail.htm?csnumber=51126
- [F] *libfixmath: Cross Platform Fixed Point Maths Library*, <http://code.google.com/p/libfixmath>
- [J] *MIL-STD-1589C (USAF) JOVIAL (J73)*, 6 July 1984, http://www.everyspec.com/MIL-STD/MIL-STD+%281500+-+1599%29/MIL-STD-1589C_14577/
- [M] John McFarlane, *Fixed-Point Real Numbers*, https://github.com/WG21-SG14/SG14/blob/master/Docs/Proposals/Fixed_Point_Library_Proposal.md
- [P] *PL/I*, Wikipedia, <http://en.wikipedia.org/wiki/PL/I>